NAAN MUDHALVAN PROJECT

PROJECT DOCUMENTATION

POCKETSMART AI

Your Smart Budget & Recommendation Assistant

Project Information	Details
Project Title	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Project Domain	Generative AI / Web Application / Budget Recommendation System
Primary Users	Individuals planning purchases, home interiors, events, and occasion-based jewelry
Core Modules	Home Interior Planner, Party Planner, Jewelry Planner
Backend	FastAPI / modular backend architecture
Frontend	HTML, CSS, JavaScript, Jinja2
AI Layer	Google Gemini 1.5 Flash Pro

Submitted by

Team Members: _____

Register No(s): _____

Department / Class: _____

Institution: _____

Faculty / Mentor: _____

Academic Year: _____

CERTIFICATE

PROJECT COMPLETION CERTIFICATE

This is to certify that the project titled “PocketSmart AI: Your Smart Budget & Recommendation Assistant” is a project documentation prepared for academic/project submission under the Naan Mudhalvan programme. The work describes the design and proposed implementation of a Generative AI based budget-aware recommendation application covering home interior planning, party planning and jewelry recommendations.

The project documentation has been prepared based on the project specification supplied for PocketSmart AI and is intended for submission, review, demonstration and evaluation.

Faculty / Mentor Signature	**Team Leader Signature**
_____	_____
Name: _____	Name: _____
Date: _____	Date: _____

DECLARATION

We hereby declare that the project documentation titled “PocketSmart AI: Your Smart Budget & Recommendation Assistant” has been prepared for academic submission. The documentation represents the project requirements, architecture, workflow, development activities, testing plan and expected system behavior described in the supplied project specification.

We further declare that confidential credentials such as API keys, passwords and secret tokens are not included in the printed documentation or public repository.

Student Name	Register Number	Signature
_____	_____	_____
_____	_____	_____
_____	_____	_____
_____	_____	_____

ACKNOWLEDGEMENT

We express our sincere gratitude to our institution, faculty members, mentors and the Naan Mudhalvan programme for providing the opportunity and guidance to undertake this project. We also acknowledge the project specification and technical guidance supplied for PocketSmart AI, which provided the functional requirements, architecture, development milestones and testing direction used in this documentation.

We thank all individuals who contributed suggestions, feedback and support during the planning and development of the project.

ABSTRACT

PocketSmart AI: Your Smart Budget & Recommendation Assistant is a Generative AI based recommendation platform designed to help users make budget-aware decisions for everyday lifestyle planning. The system accepts user preferences, budgets and contextual information and produces curated recommendations across three major domains: home interior planning, party planning and jewelry selection.

The supplied project specification describes the use of Google Gemini 1.5 Flash Pro as the multimodal AI layer. The system is designed to process text, numerical budget information and, for the Jewelry Planner, optional outfit images. The application combines a web frontend with a modular backend and recommendation services. It is intended to connect recommendation logic with product/service ecosystems such as Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO, using mock or simulated calls where necessary.

The system also includes user registration and login, session handling, recommendation history and a dashboard concept for personalized interaction. The project emphasizes modularity, budget adherence, contextual recommendations, responsive user interaction and testing with real-world scenarios.

TABLE OF CONTENTS

No.	Section
1	Introduction
2	Problem Statement
3	Project Objectives
4	Scope of the Project
5	Existing System and Proposed System
6	Functional Requirements
7	Non-Functional Requirements
8	System Architecture
9	Technology Stack
10	Module Description
11	Project Workflow
12	Database / Data and Session Design
13	API / Route Specification
14	User Interface Design
15	Implementation Plan
16	Testing and Validation
17	Test Cases
18	Expected Results and Outputs
19	Limitations and Assumptions
20	Security and Responsible Use
21	Future Enhancements
22	Conclusion
23	References / Source Basis
24	Appendix: Demo Script and Submission Checklist

1. INTRODUCTION

Managing a fixed budget while comparing many products and services can be time-consuming. The supplied project specification proposes PocketSmart AI as a cross-platform recommendation system that combines budget information, user preferences and contextual needs to generate personalized suggestions. The project focuses on three use cases: home interior planning, party budget planning and occasion-based jewelry recommendations.

The project is designed as a modular web application. The frontend collects user inputs, the backend manages requests and sessions, and the Gemini AI layer processes the context and produces structured recommendations. The supplied specification also describes authentication, recommendation history and personalized user interaction.

2. PROBLEM STATEMENT

Users often need to compare multiple products, services and price ranges while staying within a predefined budget. This is particularly difficult when planning a home setup, an event or an outfit-related purchase. A user may need to consider quantity, category, occasion, style, number of guests and available budget at the same time.

PocketSmart AI addresses this problem by providing a unified planning interface in which users enter their budget and requirements and receive AI-generated recommendations organized around the selected planning domain.

3. PROJECT OBJECTIVES

- Develop a budget-aware recommendation platform for home interiors, parties and jewelry.
- Use Gemini 1.5 Flash Pro to interpret user inputs and generate contextual recommendations.
- Provide separate planner workflows for Home, Party and Jewelry use cases.
- Support optional image input for the Jewelry Planner.
- Provide user registration, login, logout and session-related functionality.
- Maintain recommendation history and personalized interaction data.
- Provide a responsive frontend for entering requirements and viewing recommendation results.
- Validate the recommendation system using varied real-world budgets and preferences.

4. SCOPE OF THE PROJECT

The project scope is limited to the functionality described in the supplied specification. The three main planning domains are Home Interior, Party Planning and Jewelry Selection.

Domain	Inputs	Recommendation Focus
Home Interior	Budget, room type, quantities, preferences	Furniture, decor, lighting and related home items
Party Planning	Budget, guest count, event type, venue details	Catering, venue, decoration and entertainment
Jewelry Planning	Budget, occasion, style preference, optional outfit image	Occasion- and outfit-aligned jewelry options

The project specification mentions platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO as recommendation/product-service ecosystems. Where live integrations are not available, the specification explicitly permits mock API calls or simulated scraping.

5. EXISTING SYSTEM AND PROPOSED SYSTEM

Aspect	Traditional / Existing Approach	Proposed PocketSmart AI Approach
Planning	Manual comparison across multiple sites	Unified planner-based input
Budget control	User manually tracks spending	Budget supplied as a core recommendation context
Personalization	Generic search/filtering	AI-generated contextual suggestions
Multiple domains	Separate tools/sites for different needs	Home, Party and Jewelry in one platform
Image support	Usually separate visual search tools	Optional outfit image in Jewelry Planner
History	May be scattered across services	Recommendation history module

6. FUNCTIONAL REQUIREMENTS

- User registration and login.
- User logout and session management.
- Home Interior Planner with budget, room and quantity inputs.
- Party Planner with event type, budget, guest count and venue details.
- Jewelry Planner with budget, occasion, style and optional outfit image.
- Gemini AI based recommendation generation.
- Recommendation details display.
- Recommendation history retrieval.
- Dashboard concept for recent recommendations and saved queries.
- Fallback/default recommendations when AI results are insufficient.
- Responsive presentation of recommendation cards and item details.

7. NON-FUNCTIONAL REQUIREMENTS

- Usability: The interface should provide clear forms and readable recommendation outputs.
- Modularity: Routes, services and models should be separated so that components can be maintained independently.
- Reliability: Input validation and fallback handling should reduce failures from invalid or insufficient inputs.
- Performance: The application should provide timely responses for normal planner requests.
- Security: Credentials and API keys should not be hard-coded or exposed in public repositories.
- Maintainability: AI prompt logic and recommendation logic should be kept in dedicated service/utilities.
- Responsiveness: The frontend should support convenient interaction across common screen sizes.

8. SYSTEM ARCHITECTURE

The supplied specification describes a layered architecture consisting of a web frontend, a backend service layer, a Gemini AI layer and product/service data sources. The frontend collects budgets, preferences and optional images. The backend routes requests to the appropriate planner and manages sessions. The Gemini layer interprets the request and generates recommendations. Product/service sources are represented through third-party APIs or mock/simulated integrations.

Layer	Responsibility
Frontend UI	Collects user inputs and displays recommendations using HTML/CSS/JavaScript/Jinja2 templates.
Backend	Provides routing, request handling, authentication/session control and planner endpoints.
AI Layer	Uses Gemini 1.5 Flash Pro for contextual recommendation generation and multimodal processing.
Recommendation Services	Handles prompt orchestration, budget formatting, domain segmentation and recommendation logic.
Data / External Sources	Represents product/service information from platforms such as Amazon, Flipkart, IKEA, Zomato and others; mock/simulated integration is permitted by the specification.

Logical flow: User → Frontend Form → Backend Route → Planner/Recommendation Service → Gemini AI → Recommendation Response → Frontend Result Page.

9. TECHNOLOGY STACK

Technology / Tool	Purpose
Python	Backend and AI integration programming
FastAPI	Backend API framework described in the later milestones
Flask	Referenced in the initial architecture/core functionality description
Gemini 1.5 Flash Pro	Generative AI and multimodal recommendation layer
HTML / CSS / JavaScript	Frontend interface and dynamic result presentation
Jinja2	Template rendering
JWT	Token-based authorization route described in the specification
Uvicorn	Server startup for the FastAPI application
.env	Environment configuration for secrets/settings

Documentation accuracy note: The supplied project document uses both Flask and FastAPI terminology. The architecture section initially describes a Flask backend, while Milestone 3 explicitly describes a FastAPI-powered backend and main.py. This documentation therefore records both as they appear in the source rather than silently changing the project specification. The implementation should follow the framework actually used by the student team.

10. MODULE DESCRIPTION

Module	Description
Authentication	Registration, login, logout, JWT token and session-related operations.
Home Planner	Collects home budget, room details and quantities and

	generates home-related recommendations.
Party Planner	Collects event type, budget, guest count and venue information and generates event recommendations.
Jewelry Planner	Collects budget, occasion and style preferences and optionally processes an outfit image.
Recommendation Engine	Coordinates prompts, budget context, domain segmentation and AI responses.
Dashboard	Displays recent recommendations, saved queries and personalized insights.
History	Retrieves previous recommendation queries and results.
Frontend	Provides pages, forms, navigation and recommendation result cards.

11. PROJECT WORKFLOW

1. Set up the Gemini API environment and validate text and image connectivity.
2. Create the backend structure and planner routes.
3. Implement Home, Party and Jewelry recommendation workflows.
4. Implement authentication and session management.
5. Connect frontend forms with backend POST routes.
6. Render AI responses as structured recommendation cards.
7. Test the system with varied real-world budgets and preferences.
8. Refine prompts, validations, session handling and fallback behavior.
9. Document the completed project and demonstrate the working system.

12. DATABASE / DATA AND SESSION DESIGN

The supplied project specification does not define a specific relational database schema. It does define user registration, authentication, session metadata, session-specific data, recommendation history and saved/personalized interactions. Therefore, the following is a logical data design for documenting the specified behavior, not a claim that a particular database engine is required.

Logical Entity	Important Data
User	User ID, registration credentials, authentication-related data
Session	User ID, login status, session metadata
Recommendation Request	User ID, planner type, budget, preferences, contextual inputs
Recommendation Result	Generated recommendations, category details and output data
History	Previous recommendation queries and results

13. API / ROUTE SPECIFICATION

Route	Purpose
/register	Handles new user registration.
/login	Authenticates existing users and initiates a session.
/logout	Terminates the current user session.
/token	Issues a JWT token after successful authentication.
/session-info	Retrieves current session metadata.

/session-data	Returns session-specific data for personalization and recommendation tracking.
/generate-home	Generates home interior recommendations.
/generate-party	Generates party planning recommendations.
/generate-jewelry	Generates jewelry recommendations using text and optional image input.
/recommendations-details	Returns detailed AI-generated recommendations for a selected category.
/history	Retrieves past recommendation queries and results.
/startup	Initializes services/configuration when the server starts.

14. USER INTERFACE DESIGN

Page	Purpose
Home Page	Introduces PocketSmart AI and provides navigation to planner modules and user actions.
Register Page	Allows new users to create an account.
Login Page	Authenticates existing users.
Dashboard	Displays recent recommendations, saved queries and personalized insights.
Home Interior Planner	Accepts home budget, room and decor-related requirements.
Home Recommendations	Displays AI-generated home product suggestions.
Party Planner	Accepts event, budget and guest information.
Party Recommendations	Displays food, venue and decoration recommendations.
Jewelry Planner	Accepts occasion, budget, style and optional outfit image.
Jewelry Recommendations	Displays personalized jewelry options.
History	Displays previous recommendation queries and results.

15. IMPLEMENTATION PLAN

Milestone	Activities	Deliverable
1. Gemini AI Initialization	API setup, permissions, connectivity tests, prompt preparation	Working AI API connection
2. Core Functionality	Planner routes, AI utilities, authentication and modular structure	Backend planner functionality
3. FastAPI Integration	Routes, models, CORS, session and startup handling	Integrated API backend
4. UI Development	HTML forms, Jinja2 templates, recommendation cards	Working frontend
5. Testing & Optimization	Real-world scenarios, validation, prompt refinement, fallback handling	Validated and improved system

PROJECT DEVELOPMENT EVIDENCE – 8 PHASES

The following visual evidence has been added to document the actual development and execution of PocketSmart AI. The screenshots were supplied from the working project environment and application. They are

placed according to the eight development phases used for the project report. For security, the API key visible in the original environment screenshot has been redacted in this report.

PHASE 1 – BRAINSTORMING & IDEATION

The project concept is centered on a unified budget-planning assistant with separate Home, Party and Jewelry planners. The dashboard screenshot demonstrates this intended product structure through the three planner modules and recent activity area.

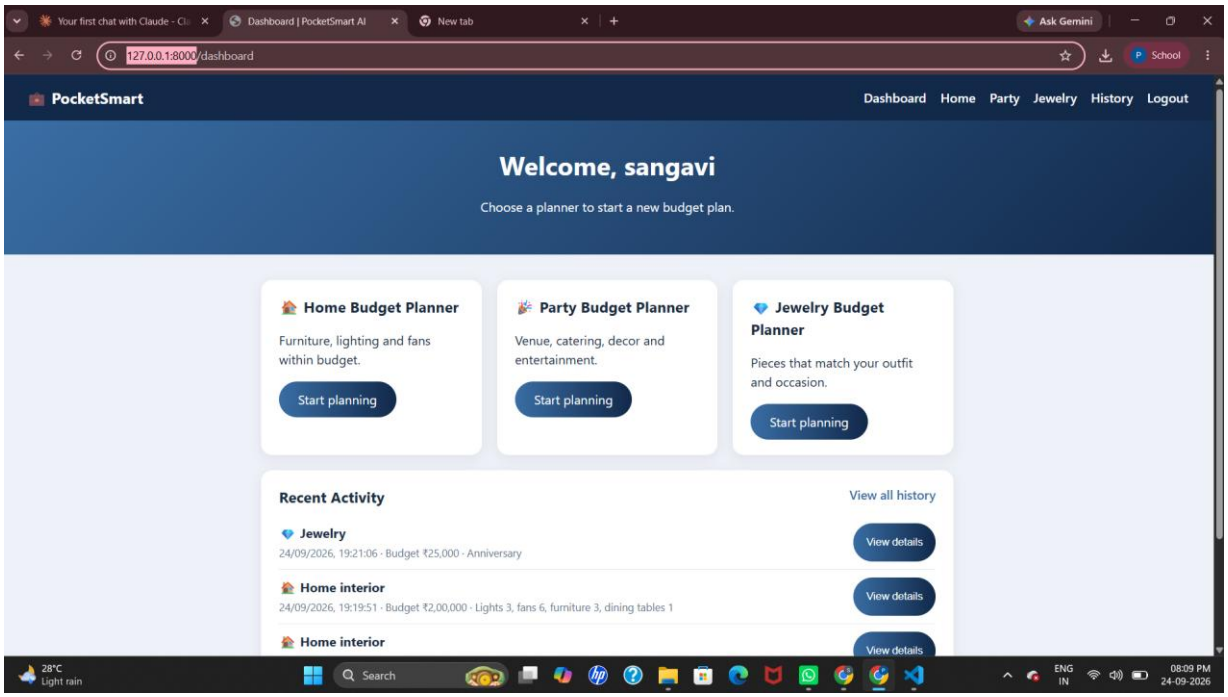


Figure 1. PocketSmart AI dashboard showing the three planner modules and recent activity.

PHASE 2 – REQUIREMENT ANALYSIS

The Home Planner interface demonstrates the concrete information requirements collected from the user, including total budget, lighting/fixture quantity, ceiling fans, furniture pieces, dining tables, rooms and special requirements.

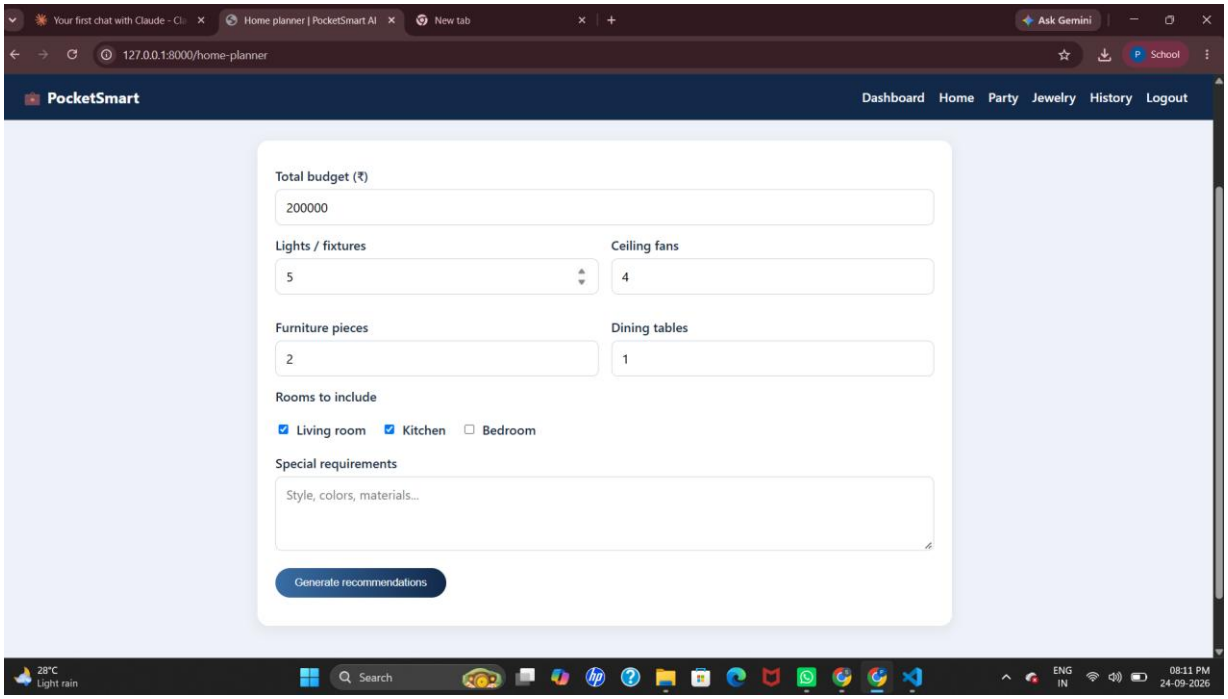


Figure 2. Home Planner input interface used to capture budget and planning requirements.

PHASE 3 – PROJECT DESIGN

The development setup screen documents the FastAPI application execution process, including the Python environment, server startup command and local application/API documentation address used during development.

```
## Setup in VS Code
1. Install Python 3.10+ and VS Code with the "Python" extension.
2. VS Code: File > Open Folder > select the `PocketSmart` folder.
3. Terminal > New Terminal, then:
   - `python -m venv venv`
   - Windows: `venv\Scripts\activate` (PowerShell: if blocked, run `Set-ExecutionPolicy
   -Scope Process Bypass` first)
   - macOS/Linux: `source venv/bin/activate`
   - `pip install -r requirements.txt`
4. Press Ctrl+Shift+P, choose "Python: Select Interpreter", pick the one inside `venv`.
5. Get a free key at https://aistudio.google.com (Get API key > Create API key).
6. Copy `.env.example` to `.env` (Windows: `copy .env.example .env`) and paste your key
into `GEMINI_API_KEY`.
   Optionally set `GEMINI_MODEL` (default `gemini-2.5-flash`) and a random `SECRET_KEY`.

## Run
`python main.py`  (or `uvicorn main:app --reload`)  then open http://127.0.0.1:8000
Interactive API docs: http://127.0.0.1:8000/docs

## Test
1. Register at /register, then sign in.
2. Home planner: submit the default form; check the summary, category tables and shopping
```

Figure 3. Development environment and application run configuration for PocketSmart AI.

PHASE 4 – PROJECT PLANNING

The dependency configuration records the principal software libraries required by the application, including FastAPI, Uvicorn, Jinja2, PyJWT, bcrypt, python-dotenv, Google GenAI and Pillow.

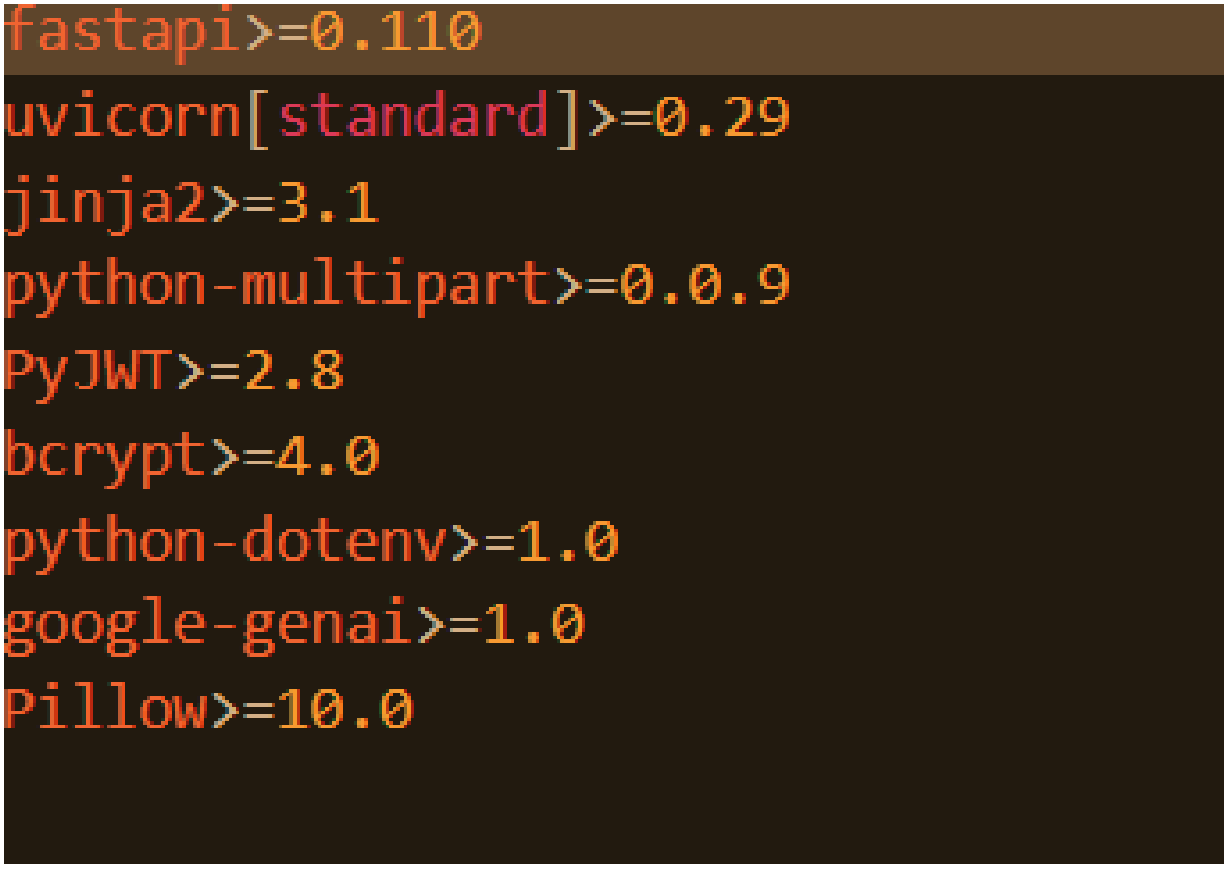


Figure 4. Project dependency configuration used for environment planning and reproducibility.

PHASE 5 – PROJECT DEVELOPMENT

The working registration page provides evidence of the implemented authentication interface. The dashboard then connects authentication with the main planner modules, while the Home Planner provides the user-input workflow for recommendation generation.

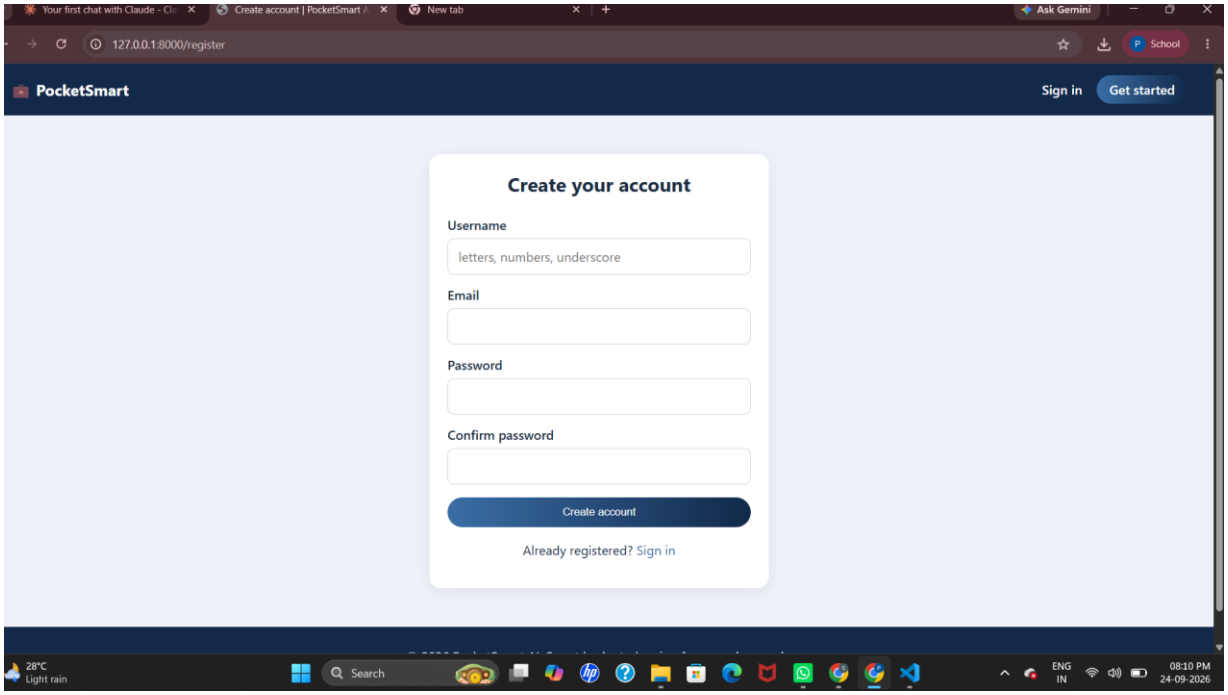


Figure 5. Implemented PocketSmart registration page.

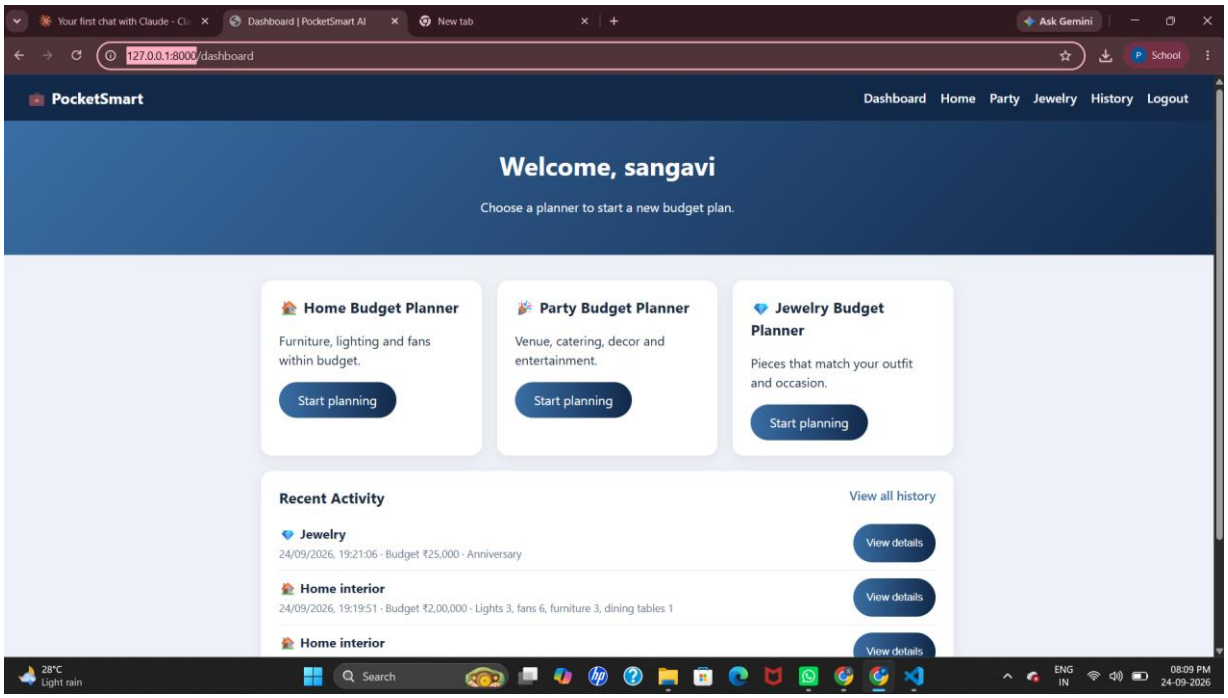


Figure 6. Implemented dashboard connecting authentication, planners and recommendation history.

PHASE 6 – PROJECT TESTING

The running application was tested through the browser using the local FastAPI server. The dashboard displays recent activity, while the Home Planner accepts varied budget and quantity inputs before submitting a recommendation request. These screens provide visible evidence that the user workflow is operational.

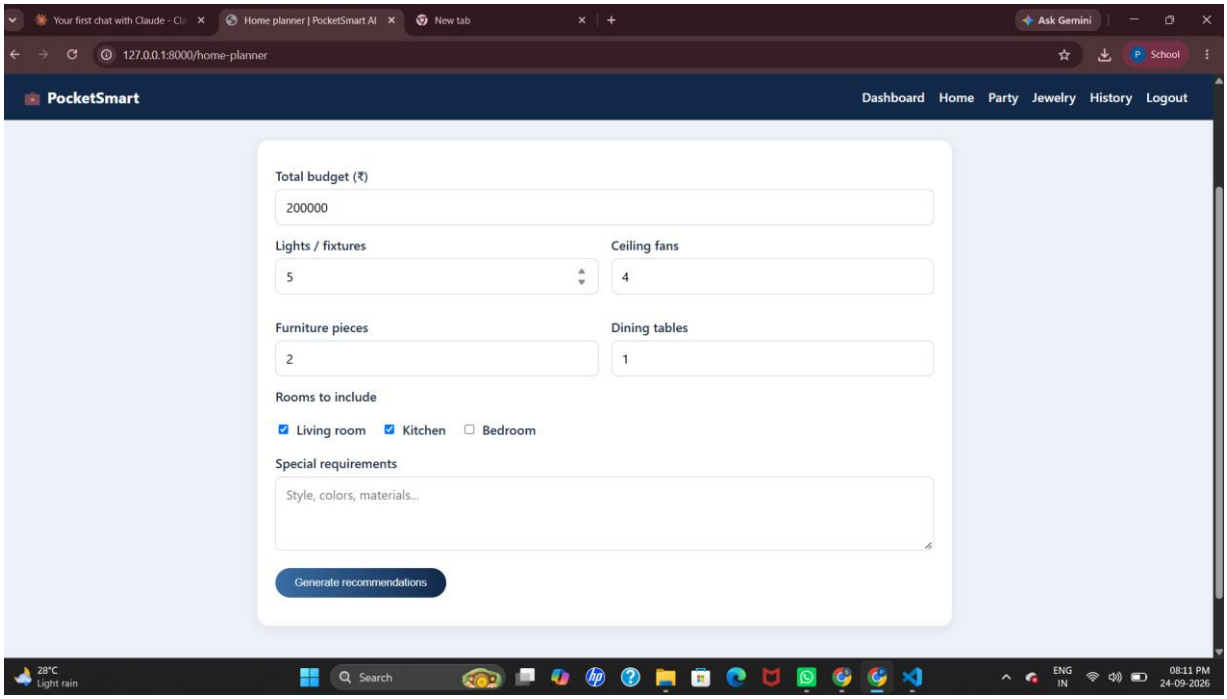


Figure 7. Home Planner test input screen used during application validation.

PHASE 7 – PROJECT DOCUMENTATION

The project setup documentation records environment creation, dependency installation, API-key configuration through environment variables, server startup and local API documentation access. The environment screenshot included here has been deliberately redacted so that no real credential is reproduced in the printed report.

```
GEMINI_API_KEY [API KEY REDACTED FOR DOCUMENTATION]
SECRET_KEY=change-this-to-a-long-random-string
```

Figure 8. Redacted environment configuration example showing model and secret-variable setup.

PHASE 8 – PROJECT DEMONSTRATION

The demonstration flow begins with user registration, continues to the authenticated dashboard, and proceeds to the planner interface. The following screenshots document the visible application journey used for the project demonstration.

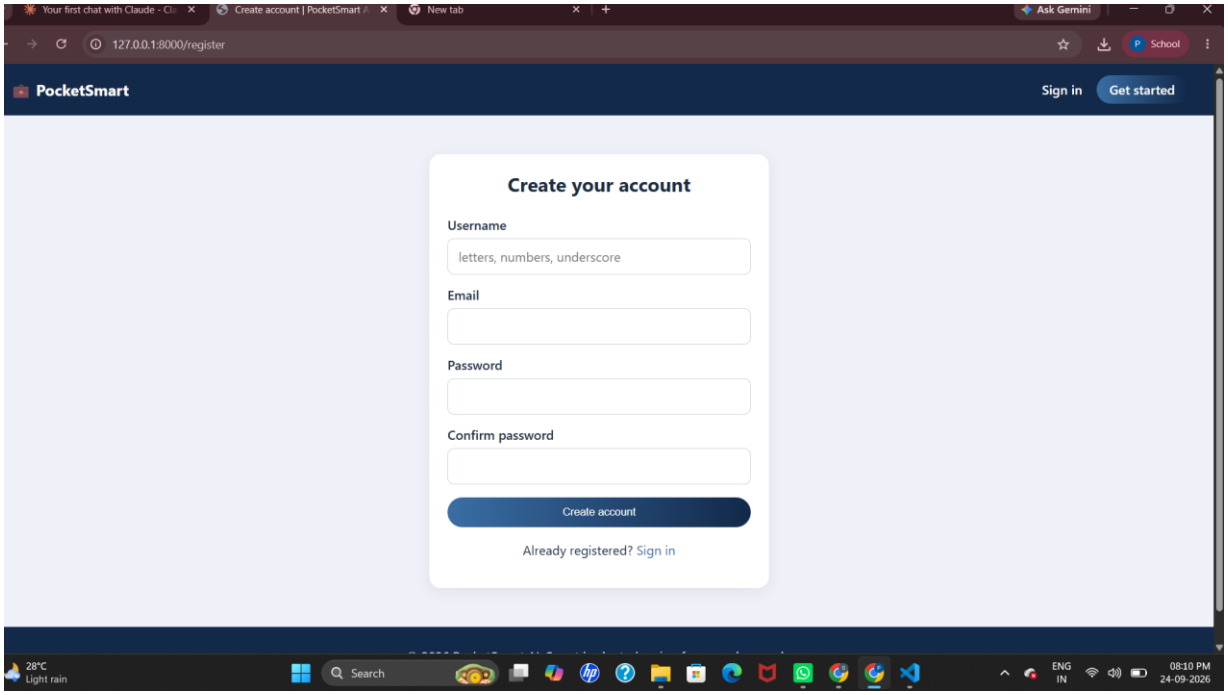


Figure 9. Demonstration – user registration page.

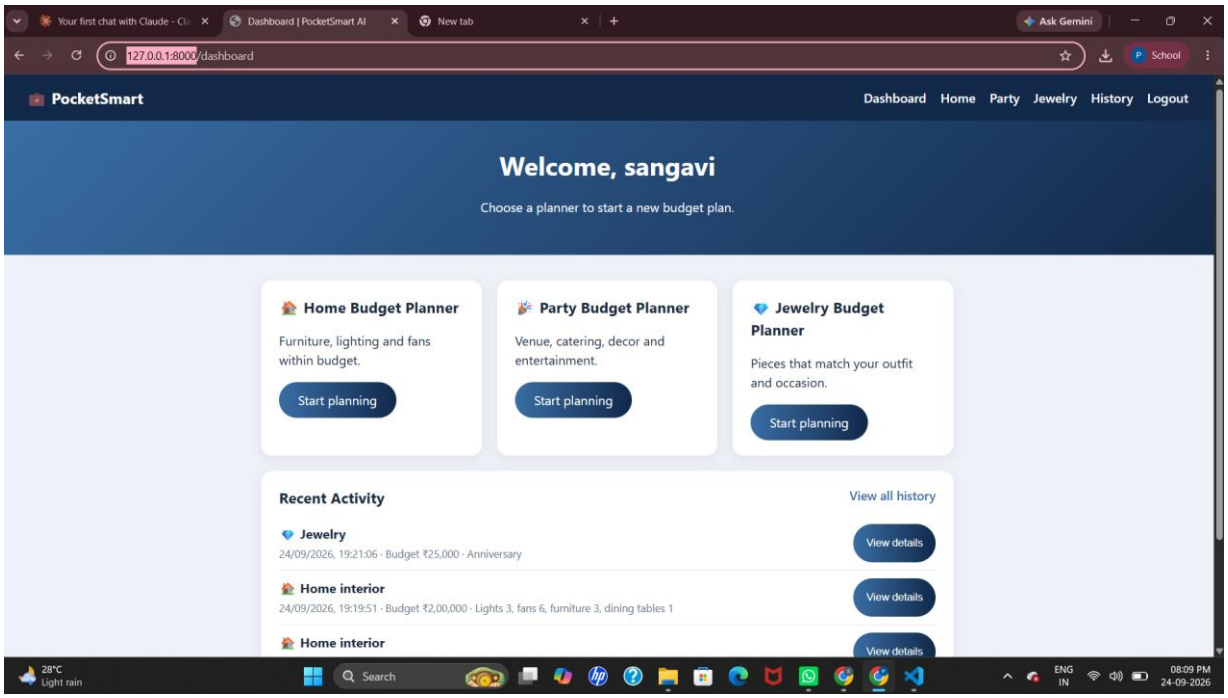


Figure 10. Demonstration – authenticated dashboard.

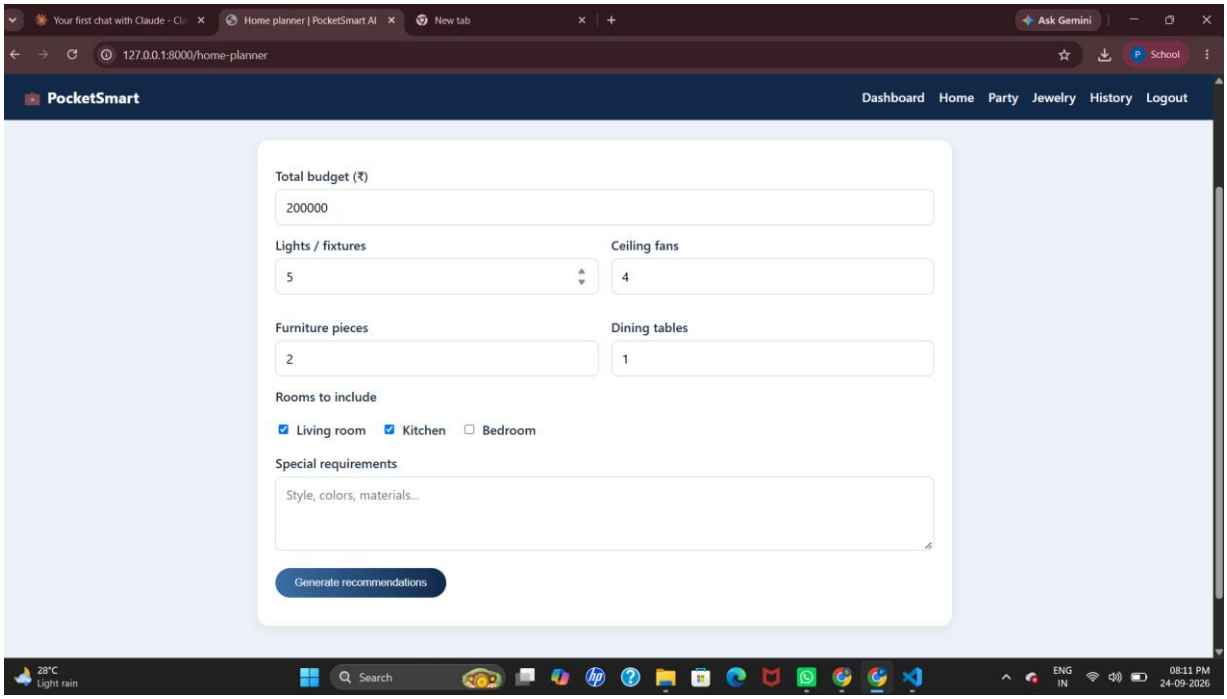


Figure 11. Demonstration – Home Budget Planner.

Note: The screenshots above are implementation and execution evidence supplied for this documentation. They should be retained as supporting evidence in the final printed submission.

16. TESTING AND VALIDATION

Testing should cover functional correctness, input validation, AI response quality, budget adherence, platform/reference accuracy where applicable, authentication/session behavior, image handling and frontend usability. The supplied specification specifically calls for real-world tests across home decor, party packages and jewelry suggestions.

Testing Area	Validation Approach
Home Planner	Use different budgets, room types, quantities and preferences; verify relevant recommendations.
Party Planner	Vary guest count, event type, venue and budget; verify category-wise suggestions.
Jewelry Planner	Use different occasions/styles and test both text-only and image-assisted requests.
Budget Adherence	Check whether recommendations remain aligned with the supplied budget context.
Authentication	Verify registration, login, logout and protected/session-dependent behavior.
History	Verify previous recommendation queries/results can be retrieved.
Invalid Inputs	Test missing, empty, negative or inconsistent budget/quantity inputs and confirm validation.
AI Failure	Simulate insufficient AI results and verify fallback/default recommendation behavior.
UI	Verify forms, results, navigation and layouts on common screen sizes.

17. TEST CASES

ID	Test	Input	Expected Result	Status
TC01	Register with valid details	Valid user information	Account created successfully	Pass/Fail
TC02	Login with valid credentials	Registered username/password	User authenticated and session initiated	Pass/Fail
TC03	Logout	Authenticated user	Session terminated and redirected appropriately	Pass/Fail
TC04	Generate Home recommendation	Budget + room + quantities	Home recommendations displayed	Pass/Fail
TC05	Generate Party recommendation	Budget + guests + event type	Party recommendations displayed	Pass/Fail
TC06	Generate Jewelry recommendation	Budget + occasion + style	Jewelry recommendations displayed	Pass/Fail
TC07	Jewelry image input	Valid outfit image + text inputs	Image-aware recommendation response	Pass/Fail
TC08	Invalid budget input	Missing/invalid budget	Validation/error handling	Pass/Fail
TC09	AI insufficient response	Simulated weak/empty response	Fallback recommendation shown	Pass/Fail
TC10	Recommendation history	Previously generated result	History entry retrieved	Pass/Fail
TC11	Session data	Authenticated user	Correct session information returned	Pass/Fail
TC12	Recommendation details	Valid recommendation request	Detailed result returned	Pass/Fail

18. EXPECTED RESULTS AND OUTPUTS

The expected system output is a set of structured, budget-aware recommendations for the selected planner. For Home, the output should contain suitable furniture, decor or lighting suggestions. For Party, it should organize possible catering, venue and decoration/entertainment options. For Jewelry, it should provide occasion- and style-aligned suggestions and may use the uploaded outfit image as contextual input.

The project specification also describes pages for authentication, dashboard, planner input, recommendation output and recommendation history. These pages together form the intended end-to-end user journey.

19. LIMITATIONS AND ASSUMPTIONS

- The supplied specification does not provide a complete database schema or a single finalized database technology.
- The specification references both Flask and FastAPI; the actual implementation should be documented according to the framework used by the student team.

- Third-party product/service integrations may be mock or simulated as explicitly permitted in the project workflow.
- AI-generated recommendations may require validation before real-world purchase or event decisions.
- Availability, price and product information from external platforms may change and should not be treated as permanently fixed data.
- The printed report should not contain real API keys, passwords, JWT secrets or other credentials.

20. SECURITY AND RESPONSIBLE USE

- Store API keys and secrets in environment configuration rather than source code.
- Do not commit .env files containing live credentials to a public GitHub repository.
- Use authentication and authorization controls for protected routes.
- Validate and sanitize user inputs before processing.
- Restrict access to user-specific session/history data.
- Treat uploaded images and user-provided information as private application data.
- Do not present AI-generated recommendations as guaranteed factual or real-time product availability unless verified through the relevant source.

21. FUTURE ENHANCEMENTS

- Integrate verified live product/service APIs where permitted.
- Add richer price comparison and filtering.
- Add budget allocation charts and visual spending breakdowns.
- Add more planning domains such as travel, grocery planning or education-related purchases.
- Improve recommendation explainability by showing why an item fits the user's budget and preferences.
- Add stronger persistence for saved plans and recommendation history.
- Add deployment and monitoring for production use.
- Improve multimodal analysis for additional planning scenarios.

22. CONCLUSION

PocketSmart AI is designed as a practical Generative AI application for budget-aware lifestyle planning. The project combines a web interface, modular backend services and Gemini-based recommendation generation to support Home Interior, Party Planning and Jewelry recommendation scenarios. The supplied specification also includes authentication, session management, recommendation history and a personalized dashboard concept.

The project demonstrates how generative AI can be integrated into a structured web application to interpret budgets and preferences and return contextual recommendations. Its modular architecture allows the planner modules, AI service logic and frontend components to be developed and tested independently.

23. REFERENCES / SOURCE BASIS

Primary source used for this documentation:

- PocketSmart AI: Your Smart Budget & Recommendation Assistant — supplied 39-page project specification/document.

The technical terminology, modules, workflows, routes, milestones and intended outputs in this report are based on the supplied project specification. No claim is made here that a feature has been implemented unless it is described as part of the specified project design or implementation plan.

24. APPENDIX – PROJECT DEMO SCRIPT

The following script can be used for the required project demonstration video.

Step	What to Demonstrate / Explain
1	Introduce the project name: PocketSmart AI – Your Smart Budget & Recommendation Assistant.
2	Explain the purpose: help users obtain budget-aware recommendations for home interiors, parties and jewelry.
3	Show registration/login and explain authentication if implemented.
4	Open Home Interior Planner; enter a sample budget, room and quantity requirements; generate recommendations.
5	Open Party Planner; enter event type, guest count and budget; generate recommendations.
6	Open Jewelry Planner; enter budget, occasion and style; optionally upload an outfit image; generate recommendations.
7	Show recommendation details and explain how the AI output is organized.
8	Show dashboard/history if implemented and explain saved/recent recommendations.
9	Briefly explain the architecture: frontend → backend → recommendation service → Gemini AI → output.
10	Show the final output and summarize the benefits and project scope.

APPENDIX – FINAL PRINT / SUBMISSION CHECKLIST

- ☐ Cover page completed with student/team/institution details.
- ☐ Certificate signed by the appropriate faculty/mentor.
- ☐ Declaration signed by team members.
- ☐ Table of contents checked after final pagination.
- ☐ Project screenshots inserted where required by faculty/portal instructions.
- ☐ Source code repository is public if the SmartBridge submission instruction requires a public repository.
- ☐ Repository is organized phase-wise according to the required 8 phases.
- ☐ No API keys, passwords or secret credentials are present in the public repository.
- ☐ Demo video is recorded with screen sharing and voice-over.
- ☐ Demo video explains project name, purpose, benefits, working process and final output.
- ☐ Google Drive demo-video sharing is set to Anyone with the link can view, if required.
- ☐ Printed report is proofread and signatures are collected.